

WEEKLY COLLABORATOR DISCUSSION

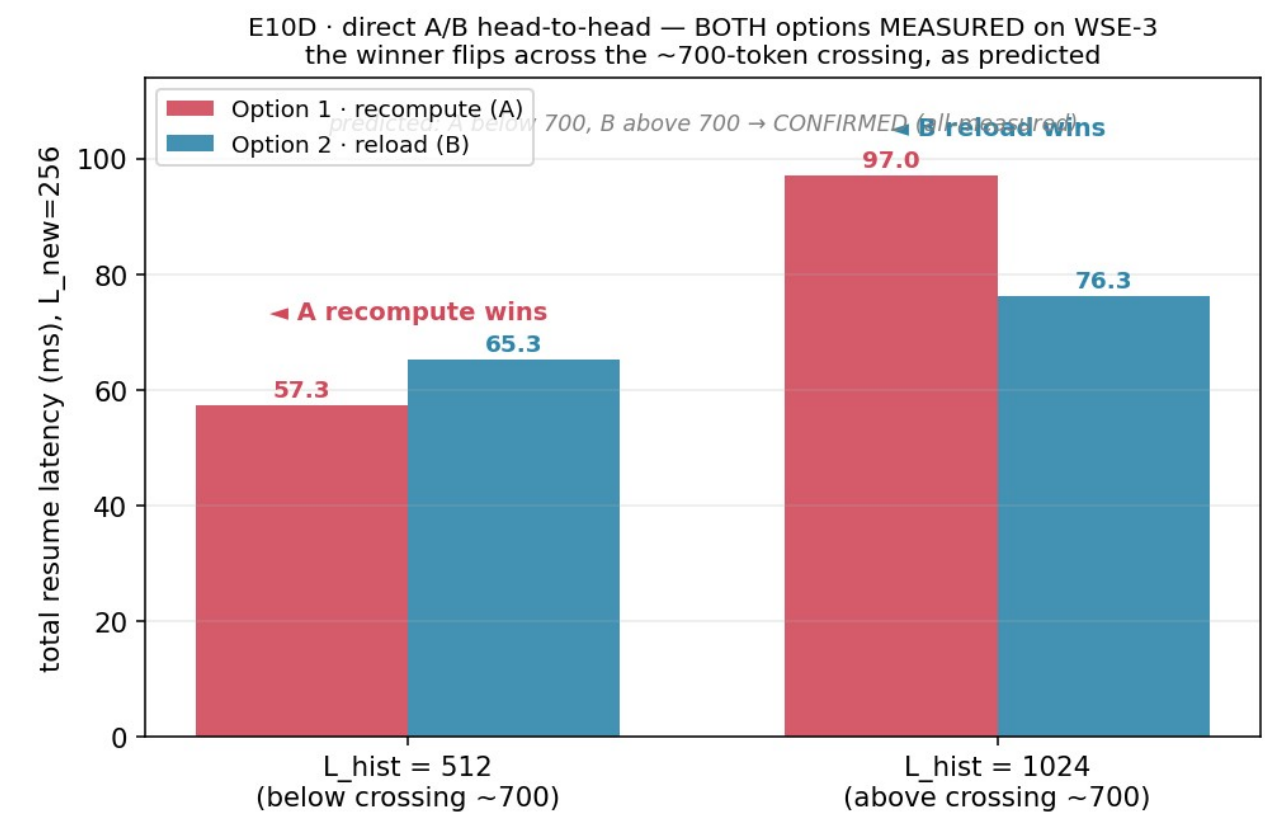
On-Chip KV Offload as a Storage Tier

Two multi-row storage designs measured on real WSE-3; the on-chip tier now sits under both prior resume options.

MODEL	EVIDENCE	SCOPE	DATE
Qwen3-1.7B geometry	81 CS-3 runs, 0-cycle spread	M3 idle-PE tier	2026-08-24

GOAL · WHERE WE ARE IN THE MILESTONES

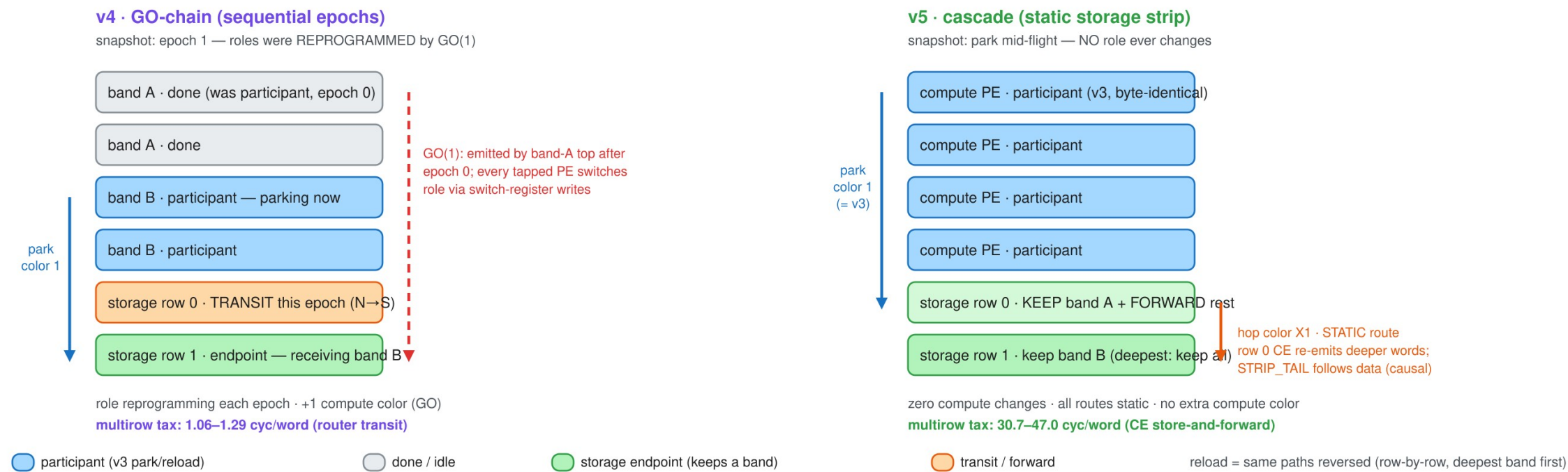
M2 ended at recompute-vs-host with a 744-token crossing; M3 asks if idle PEs beat both



Last time (M2 E10/E10D, all device-measured): below H*=744 recompute wins, above it host reload wins — but reload rides shared edge IO and the host. M3's question: can idle on-chip PEs hold parked KV as a middle tier? This week: build the multi-row storage strip both candidate ways, measure both end-to-end, and price the tier.

v4 reprograms compute roles per epoch; v5 keeps compute byte-identical and cascades inside the strip

Two multi-row designs (R=2 shown) — snapshot: parking the second half of the column



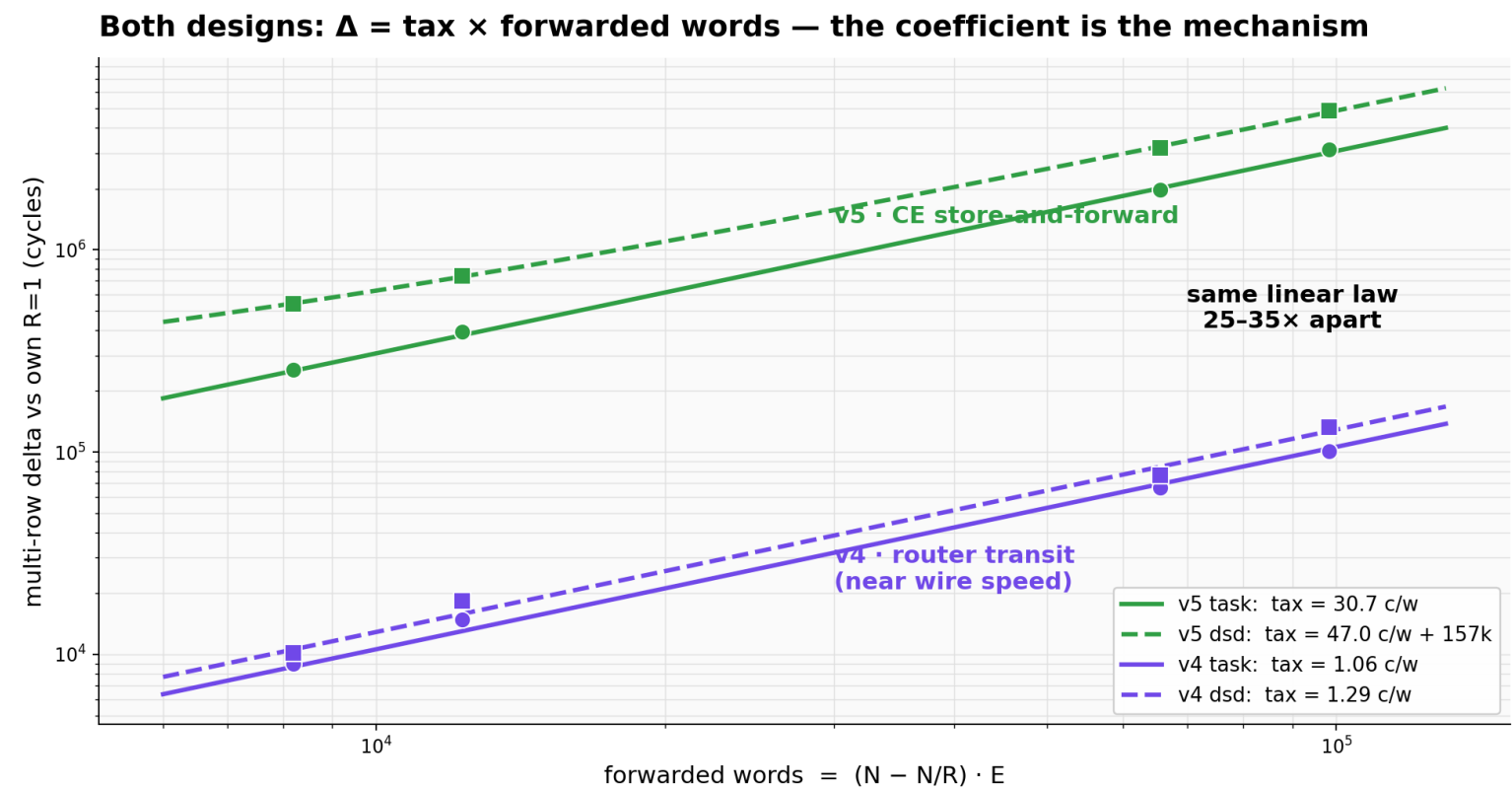
Both share the proven single-row protocol on one borrowed color. v4: bands park in sequential epochs, chained fully on-chip by a GO color; every tapped PE rewrites its switch positions per epoch. v5: row 0 keeps the nearest band and store-and-forwards deeper words on static per-hop colors; a STRIP_TAIL word rides behind the data as the causal end-of-stream. Reload returns over the same paths reversed.

CAPACITY · NOMINAL 42 KIB/STORAGE PE = 672 KV-TOKEN EQUIVALENTS PER 1×256 ROW/BLOCK

Each row adds 672-token capacity; v4 wins every multi-row setting

KV MOVED	ROWS · CAPACITY	V3 SINGLE-ROW	V4 GO-CHAIN	V5 CASCADE
task · 1,024 tok	1 · 672 tok (under)	1,388,470	1,535,920	1,420,244
task · 1,024 tok	2 · 1,344 tok (fits)	—	1,544,946	1,675,327
task · 1,024 tok	4 · 2,688 tok (fits)	—	1,550,855	1,812,526
DSD · 8,192 tok	1 · 672 tok (under)	7,350,234	8,542,434	7,482,812
DSD · 8,192 tok	2 · 1,344 tok (under)	—	8,619,349	10,669,498
DSD · 8,192 tok	4 · 2,688 tok (under)	—	8,675,026	12,343,610

Both designs obey $\Delta = \text{tax} \times \text{forwarded words}$ — the tax is the mechanism, 25-35x apart



Delta vs each design's own $R=1$ baseline collapses on the same axis, $\text{fwd_words} = (N - N/R) \cdot E$. v4's tax is router-level transit (1.06/1.29 cyc/word, near wire speed); v5's is a CE data-task per forwarded word (30.7/47.0). The v5 dsd fit needs one extra constant (~157k cycles): strip rows emit their own block before relaying, which fast owners expose.

v4 cost = endpoint baseline + router-only forwarding

1 · MODEL FORM

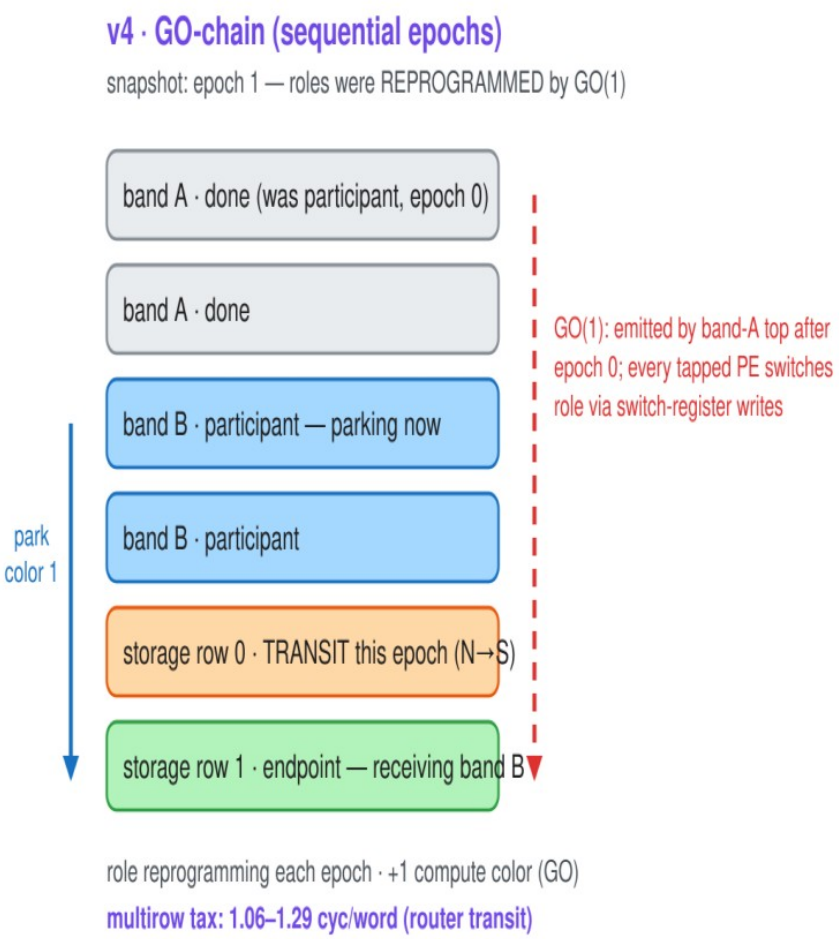
$$C_{v4}(N, E, R, m) = C_{floor}^{(m)} + (E - 4)N C_{owner}^{(m)} + W_{fwd}(N, E, R) C_{router}^{(m)}$$

$W_{fwd} = (N - N/R)E, \quad m \in \{task, DSD\}$

N = compute PEs/column · E = u32 words/PE · R = storage rows

2 · FITTED COEFFICIENTS

TERM	TASK	OWNER DSD	PHYSICAL ROLE
C_{floor}	61,058	78,676	launch/control + role setup
C_{owner}	96.00	65.08	endpoint payload, cyc/word
C_{router}	1.06	1.29	router-only fwd, cyc/word



Generic equation first, fitted coefficients second, topology at right. Setting: N=256, E>=64; pairs are task | owner-side bulk-DSD. v4 epoch_sum excludes small inter-epoch GO gaps.

The fabric is not binding: CE per-word handling dominates

WHAT 'ROUTER-ONLY' MEANS

Wavelet stays in the fabric switch

NORTH input → switch → SOUTH output

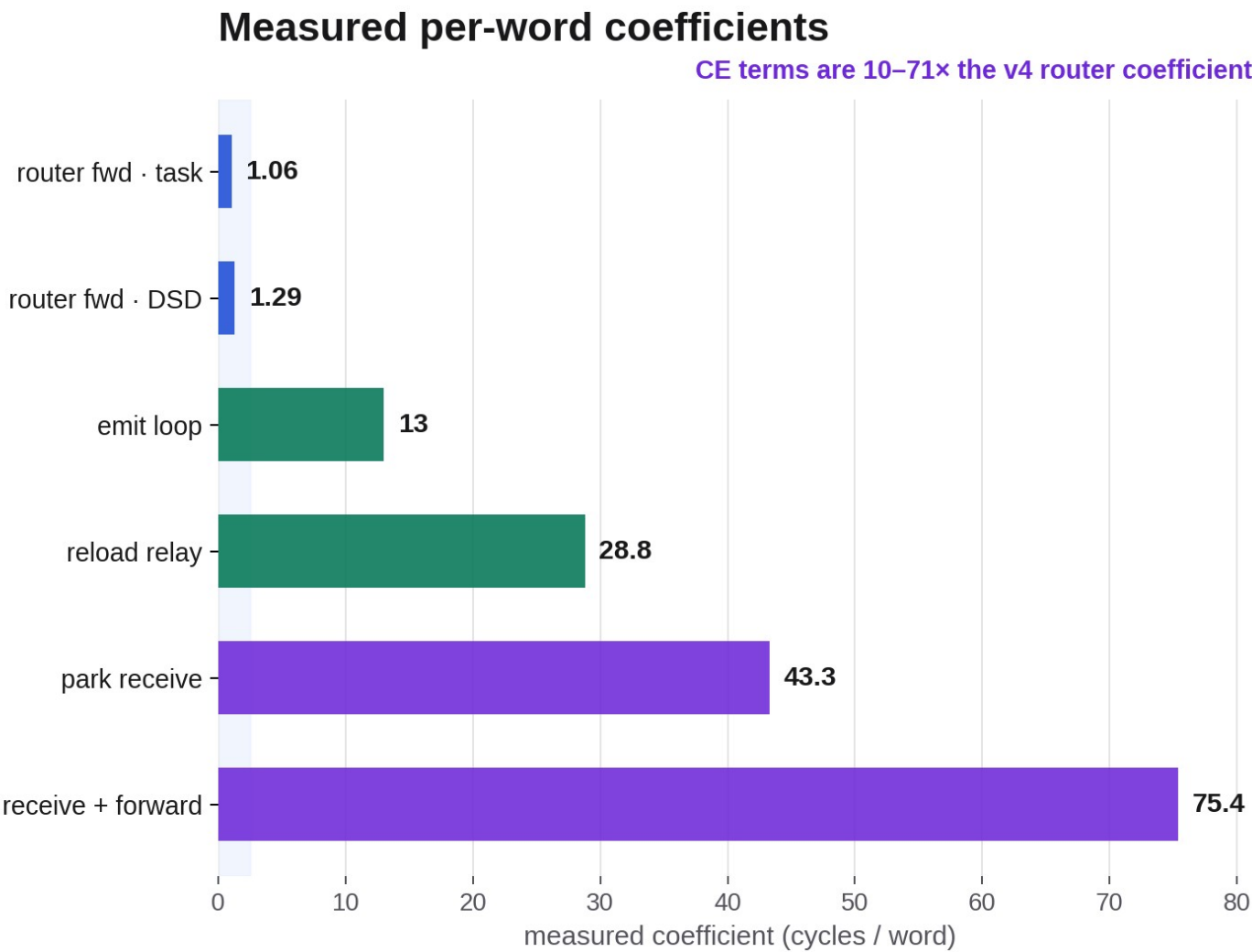
no RAMP · no IQ/task · no CE · no local copy/re-emit

Measured router-only terms

Distance latency
t_hop = 2.0 cycles/hop
round trip = 2(D-1)t_hop
+28 cycles at D=8; +124 at D=32

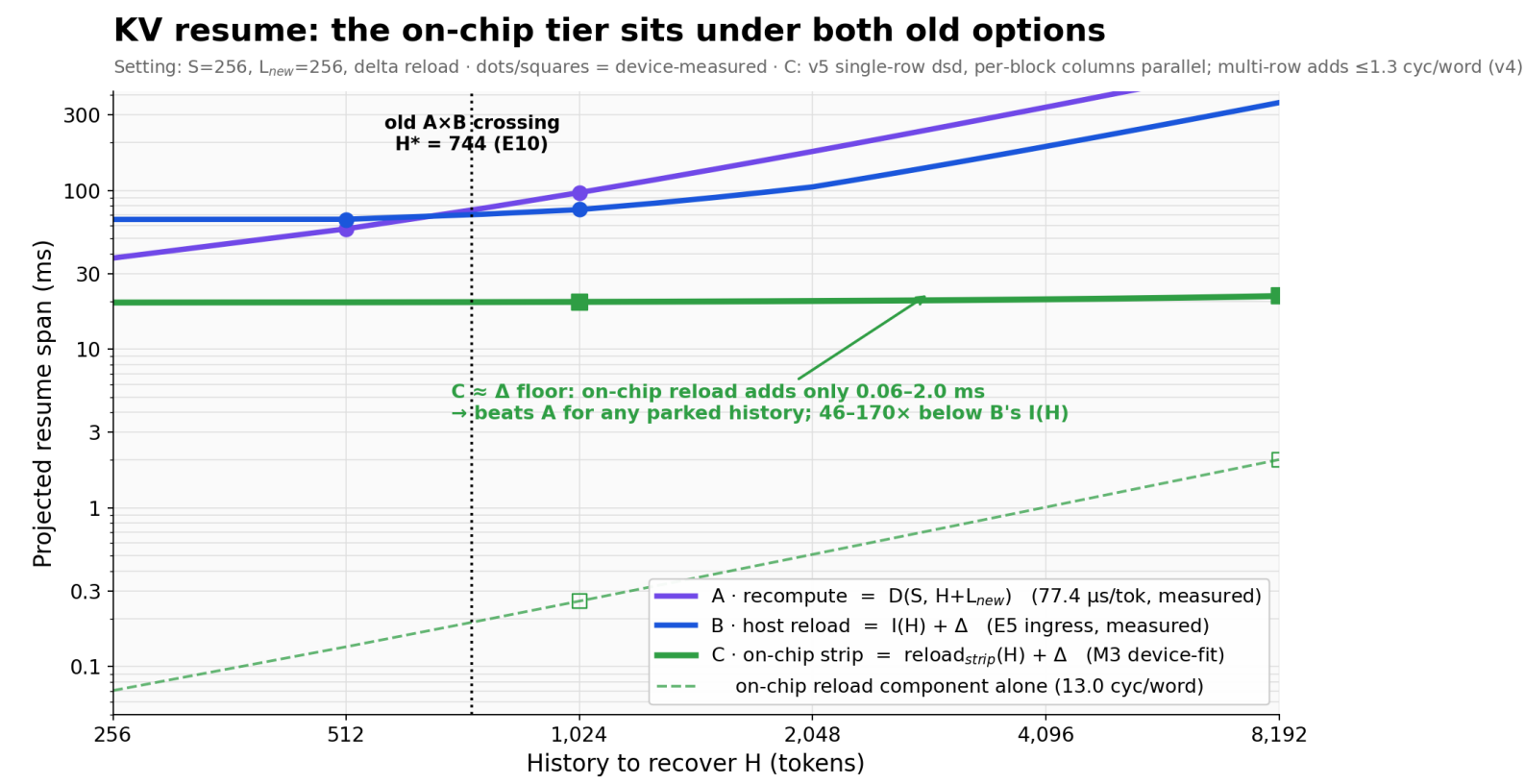
Multi-row forwarding
v4 tax = 1.06 | 1.29 cycles/forwarded-word

Still consumes fabric links and may backpressure.
Current measurements are isolated, single-lane runs.



Router-only means switch-to-switch transit with no RAMP, IQ/task, CE landing, or local copy/re-emit. It still consumes links and can backpressure; current measurements are isolated, single-lane runs.

The on-chip tier sits under both old options: resume is now delta-dominated



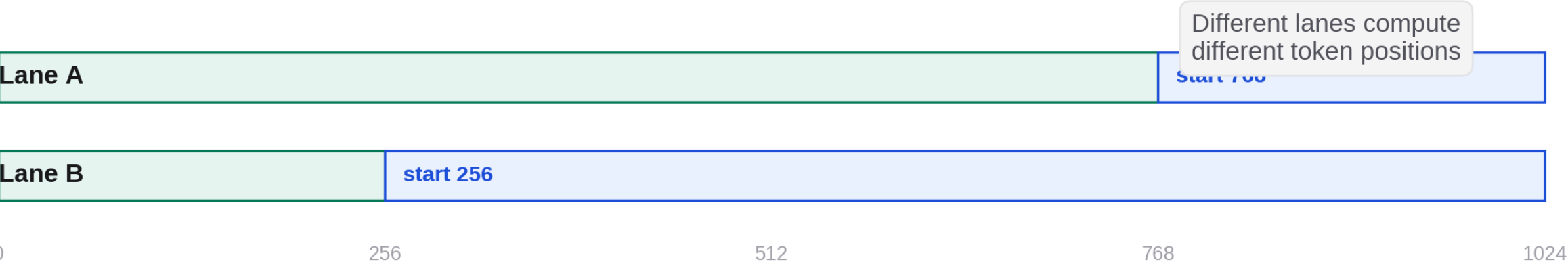
Same axes and setting as the M2 boundary figure, plus measured tier C: on-chip strip reload adds only 0.06-2.0 ms (13.0 cyc/word), 46-170x below host ingress $I(H)$, flattening resume onto the ~ 20 ms L_{new} delta floor. The old 744-token crossing becomes a fallback choice for un-parked history. Next lever for resume latency is the delta/restart path, not the transfer.

Common-start is correct—but recomputes longer prefix hits

Current: one common start for the whole batch

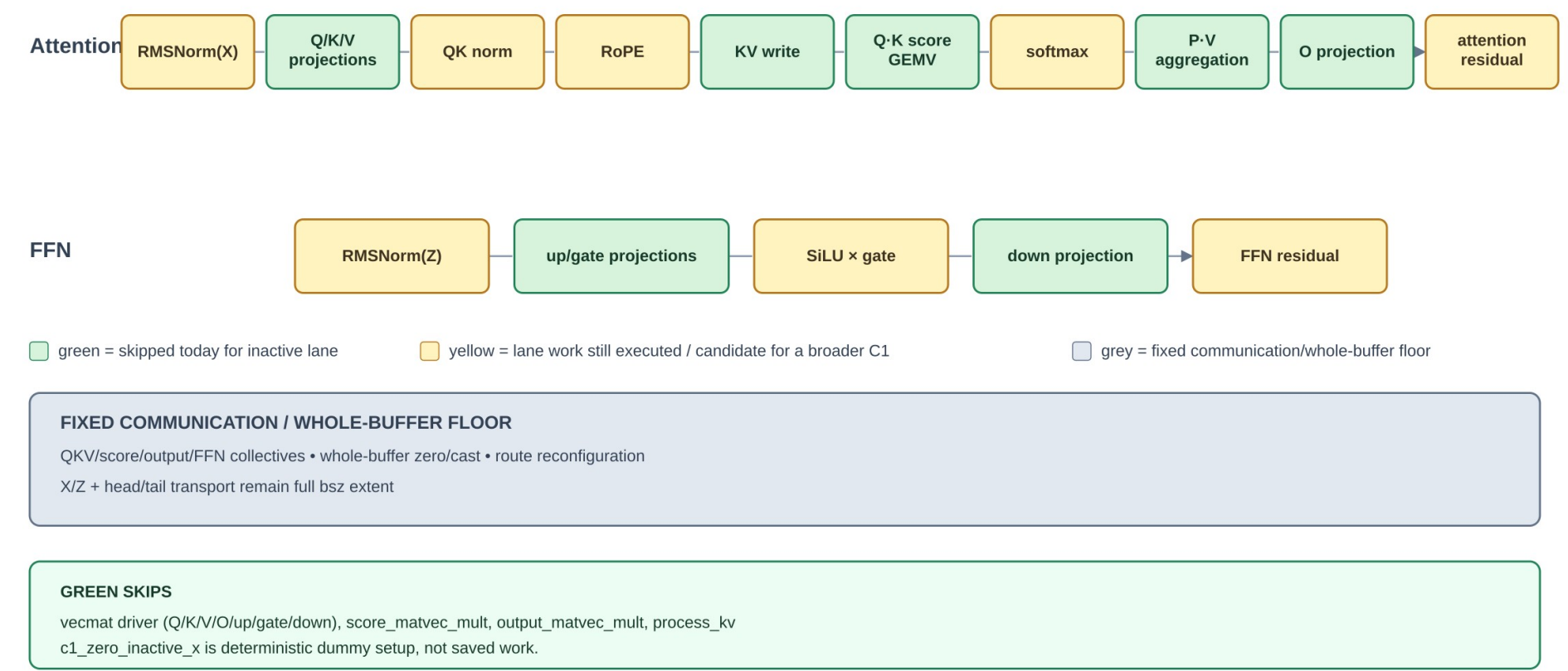


Needed: per-lane cursors inside the same round



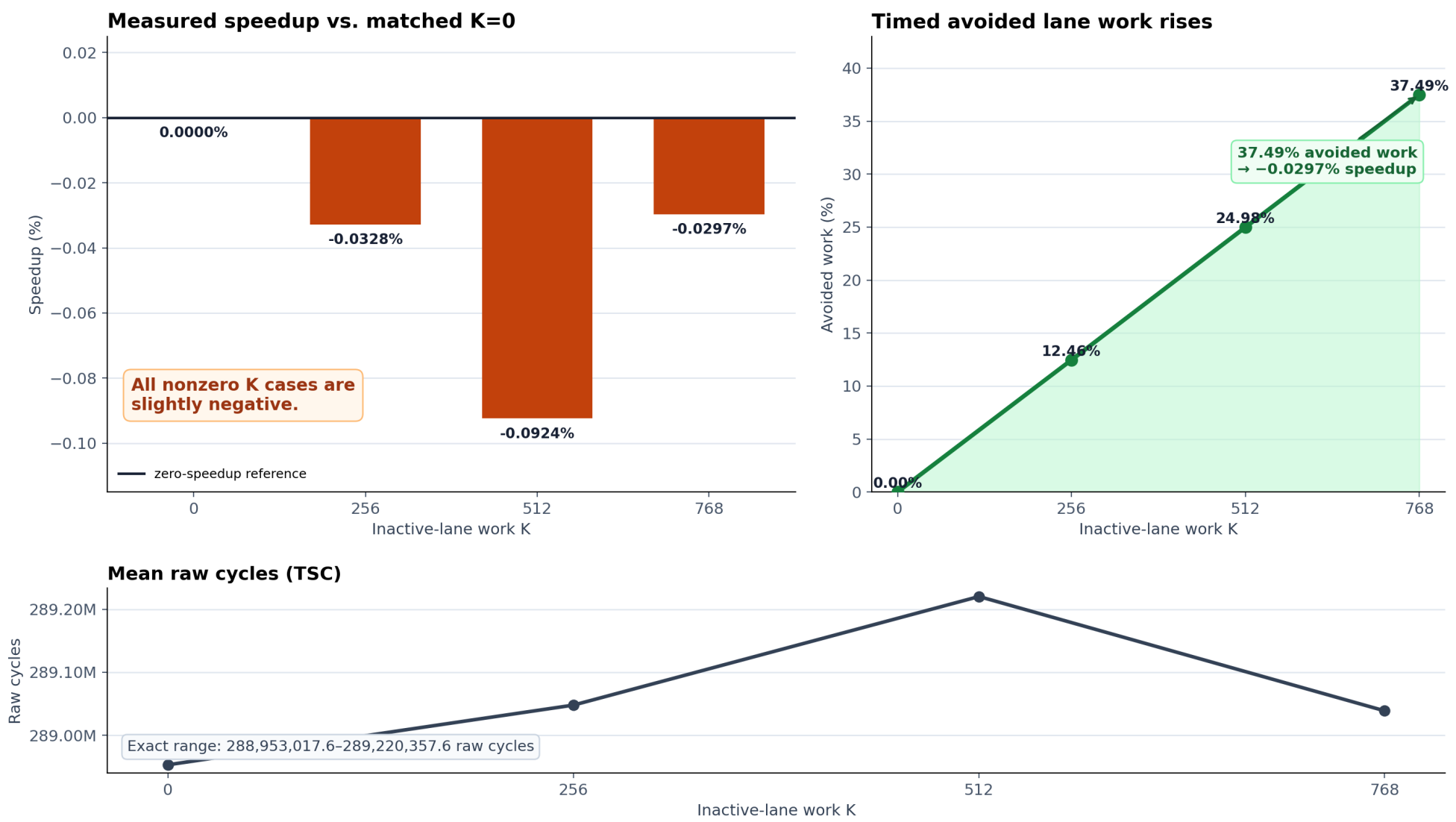
Lane A can reuse 768 tokens while Lane B can reuse only 256. The current batch starts both at 256, so Lane A re-executes 512 already-known token positions.

We skipped lane-sliced compute; the fixed batch floor stayed



Green is predicated while $\text{step} < R_i - \min(R)$; yellow still executes for the inactive lane; grey remains full-bsz communication or whole-buffer work.

37.49% less lane-step work produced no critical-path speedup



Full Qwen3-1.7B, bsz=2, S/N/F/G=256/1024/769/255, five repetitions per case, raw TSC cycles. At K=768, mean speedup is -0.0297%.

Pin the fixed floor—or move to independent lane progress

A · Profile the C1 fixed floor

- Time or ablate collectives, whole-buffer ops and transport by phase
- Identify the single largest remaining critical-path item
- Broaden masking only if that removable term can change the $<0.1\%$ result

B · Design C2 / continuous batching

- Independent per-lane start, cursor and completion state
- Price added SRAM, protocol, admission and scheduler complexity
- Only credit gains that survive a serving-like schedule replay